TRƯỜNG ĐHCN VIỆT - HUNG
KHOA CÔNG NGHỆ THÔNG TIN

ĐẠI HỌC CÔNG NGHIỆP VIỆT-HUNG

LINQ CƠ BẢN

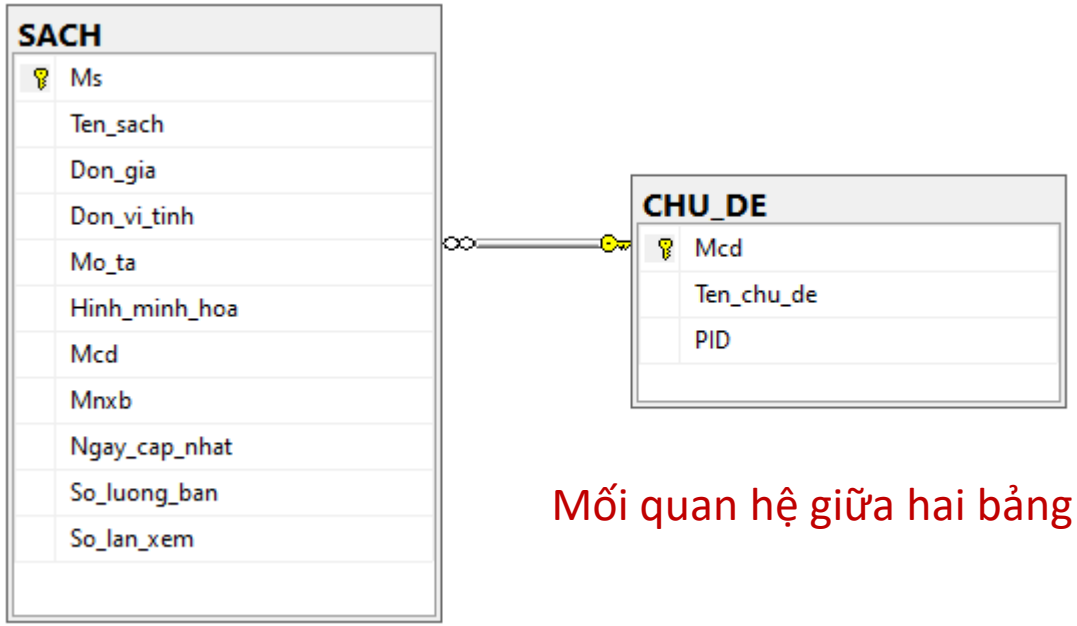
TS. Nguyễn Hồng Quân



Tổng Quan Về LINQ

- Language Integrated Query trong C# và .NET
- Đồng nhất truy vấn
  - Viết truy vấn trực tiếp bằng C# cho SQL, Objects, XML mà không cần học nhiều cú pháp riêng biệt.
- Strongly Typed
  - Kiểm tra lỗi ngay khi biên dịch (Compile-time) và hỗ trợ IntelliSense gợi ý mã nguồn tự động.
- Tích hợp EF Core
  - Tự động chuyển đổi các câu lệnh C# LINQ thành các câu lệnh T-SQL tối ưu cho SQL Server.

CSDL dùng cho các ví dụ



Mối quan hệ giữa hai bảng SACH và CHU_DE (1 - N)

- **Bảng CHU_DE:** Quản lý thể loại (Mcd: Khóa chính, Ten_chu_de, PID).
- **Bảng SACH:** Lưu trữ thông tin sách (Ms: Khóa chính, Ten_sach, Don_gia, Mcd: Khóa ngoại...).
- **Mối quan hệ:** Một chủ đề có thể chứa nhiều sách (`CHU_DE 1 - N SACH`).

Khai báo lớp entity C#

❖ Ánh xạ bảng CHU_DE và SACH sang Object trong EF Core

```
public class ChuDe {
        public int Mcd { get; set; }
        public string TenChuDe { get; set; }
        public int? PID { get; set; }
        // Navigation Property: 1 chủ đề chứa nhiều sách
        public ICollection<Sach> Sachs { get; set; }
}
```

```
public class Sach {
        public int Ms { get; set; }
        public string TenSach { get; set; }
        public decimal DonGia { get; set; }
        public int Mcd { get; set; } // Foreign Key
        public int SoLuongBan { get; set; }
        ……
        // Navigation Property đến bảng CHU_DE
        public ChuDe ChuDe { get; set; } }
```

Hai cú pháp LINQ

- Query Syntax - Gần giống với SQL truyền thống, dễ đọc với các câu truy vấn phức tạp.

```
from s in db.Sachs where s.DonGia > 100000 select s;
```

- Method Syntax (Extension Methods) - Sử dụng Lambda Expression (`=>`), phổ biến và linh hoạt trong ASP.NET Core.

```
db.Sachs .Where(s => s.DonGia > 100000) .ToList();
```

Cơ chế thực thi trong LINQ

❖Hai cơ chế thực thi:

- **Deferred Execution**: Khi khởi tạo `Where`, `Select`, câu SQL chưa chạy xuống CSDL cho đến khi duyệt `foreach` hoặc gọi `.ToList()`.

```
// Chưa gọi SQL xuống CSDL
var query = _context.Sachs.Where(s => s.DonGia > 50000);
// Câu T-SQL chính thức được gửi đi tại câu lệnh này!
var list = await query.ToListAsync();
```

- **Immediate Execution**: Các hàm trả về giá trị hoặc tập hợp như `ToList()`, `Count()`, `First()` sẽ gọi SQL ngay lập tức.

Chú ý! Cơ chế đồng bộ và Bất đồng bộ:
- Chạy đồng bộ (Không dùng await / Dùng ToList())
- Chạy bất đồng bộ (Có await / Dùng ToListAsync())

Lọc Dữ Liệu Với `Where()`

- Sử dụng điều kiện để lọc danh sách kết quả
- VD: Lấy tất cả các sách có đơn giá lớn hơn 150,000 VNĐ.

```
// Method Syntax
var dsSachGiaCao = await _context.Sachs .Where(s => s.DonGia > 150000)
.ToListAsync();

// T-SQL Tương đương: // SELECT * FROM SACH WHERE Don_gia > 150000
```

Kết hợp nhiều điều kiện lọc

- Sử dụng toán tử logic `&&` (AND) và `||` (OR)
- VD: Lấy các sách thuộc Chủ đề `Mcd = 3` VÀ có số lượng bán > 100.

```
var ketQua = await _context.Sachs .Where(s => s.Mcd == 3 && s.SoLuongBan > 100) .ToListAsync();
// Hoặc nối chuỗi nhiều phương thức Where()
var ketQua2 = await _context.Sachs .Where(s => s.Mcd == 3) .Where(s => s.SoLuongBan > 100) .ToListAsync();
```

Tìm Kiếm Chuỗi Với `Contains()`

- Tương đương mệnh đề LIKE '%tu_khoa%' trong SQL
- VD: Tìm kiếm các sách có chứa từ khóa "Lập trình" trong tên sách.

```
string tuKhoa = "Lập trình";
var dsTimKiem = await _context.Sachs .Where(s =>
s.Ten_sach.Contains(tuKhoa)) .ToListAsync();
// SQL Sinh ra: SELECT * FROM SACH WHERE Ten_sach LIKE N'%Lập trình%'
```

Sắp xếp dữ liệu: `OrderBy()`

❖Sắp xếp tăng dần và giảm dần trong LINQ

```
var tangDan = await _context.Sachs
.OrderBy(s => s.DonGia) .ToListAsync();
```

```
var giamDan = await _context.Sachs
.OrderByDescending(s => s.DonGia) .ToListAsync();
```

Sắp sếp nhiều cấp với `ThenBy()`

- Sắp xếp thứ cấp khi các giá trị đầu tiên bằng nhau
- Ví dụ: Sắp xếp Sách giảm dần theo `Don_gia`, nếu giá bằng nhau thì sắp xếp tăng dần theo `Ten_sach`.

```
var danhSach = await _context.Sachs .OrderByDescending(s =>
s.DonGia).ThenBy(s => s.TenSach).ToListAsync();
// SQL: ORDER BY Don_gia DESC, Ten_sach ASC
```

Phân Trang Dữ Liệu Với `Skip()` & `Take()`

- Kỹ thuật phân trang (Paging) trong Web ASP.NET Core MVC
- Công thức: Bỏ qua `(pageIndex - 1) * pageSize` bản ghi và lấy `pageSize` bản ghi tiếp theo.

```
int pageIndex = 2; // Trang số 2
int pageSize = 10; // Mỗi trang 10 sản phẩm
var pageData = await _context.Sachs .OrderBy(s => s.Ms)
.Skip((pageIndex - 1) * pageSize) .Take(pageSize) .ToListAsync();
```

Ánh xạ dữ liệu với `Select()`

- Chỉ lấy các cột dữ liệu cần thiết (Projection)
- Tối ưu: Thay vì `SELECT *`, chỉ chọn các thuộc tính cần dùng để tăng hiệu suất truy vấn.

```
// Lấy ra Anonymous Object chỉ gồm Ten_sach và Don_gia
var result = await _context.Sachs .Select(s => new {
        TenSach = s.TenSach,
        GiaBan = s.DonGia }
) .ToListAsync();
```

Ánh xạ sang ViewModel

❖Chuẩn hóa dữ liệu trả về cho Razor View trong MVC

```
// DTO class dùng hiển thị lên UI
public class SachVM {
        public string TenSach { get; set; }
        public decimal Gia { get; set; }
        public string TenChuDe { get; set; }
}
```

```
// LINQ Truy vấn trực tiếp sang ViewModel
var listVM = await _context.Sachs .Select(s => new SachVM {
        TenSach = s.TenSach,
        Gia = s.DonGia,
        TenChuDe = s.ChuDe.TenChuDe }
).ToListAsync();
```

Lấy Một Bản Ghi Duy Nhất

- So sánh `FirstOrDefault()` và `SingleOrDefault()`
  - Lấy bản ghi đầu tiên thỏa mãn. Nếu không tìm thấy trả về `null`. Tối ưu cho truy vấn tìm kiếm.

```
await _context.Sachs .FirstOrDefaultAsync(s => s.Ms == id);
```

  - Bắt buộc chỉ có tối đa 1 bản ghi thỏa mãn. Nếu có bản ghi thứ 2 sẽ phát sinh Exception!

```
await _context.Sachs .SingleOrDefaultAsync(s => s.Ms == id);
```

Kiểm tra sự tồn tại: `Any()` & `All()`

❖Trả về kiểu boolean (true/false) nhanh chóng

```
// Any(): Kiểm tra xem CÓ ÍT NHẤT 1 sách có giá > 500,000 không?
bool coSachDat = await _context.Sachs.AnyAsync(s => s.DonGia > 500000);
// All(): Kiểm tra TẤT CẢ sách đều có lượt xem > 0 phải không?
bool tatCaDaXem = await _context.Sachs .AllAsync(s => s.SoLanXem > 0);
```

Các hàm tính toán gom nhóm

- `Count()`, `Sum()`, `Average()`, `Min()`, `Max()`

```
int tongSoSach = await _context.Sachs.CountAsync();
decimal tongDoanhThu = await _context.Sachs .SumAsync(s => s.Don_gia
* s.SoLuongBan);
double giaTrungBinh = await _context.Sachs .AverageAsync(s =>
(double)s.DonGia);
decimal giaCaoNhat = await _context.Sachs .MaxAsync(s => s.DonGia);
```

Kết nối nhiều bảng bằng `Join()`

❖Thực hiện INNER JOIN giữa SACH và CHU_DE trong Query Syntax

```
var query = from s in _context.Sachs join c in _context.ChuDes on
s.Mcd equals c.Mcd select new {
        MaSach = s.Ms,
        TenSach = s.TenSach,
TenChuDe = c.TenChuDe };
var list = await query.ToListAsync();
```

Eager loading với `Include()`

- Cách hiện đại để lấy bảng liên quan trong EF Core thay cho Join
- Eager Loading (Tải trước / Nạp sớm) là một kỹ thuật trong ORM (như Entity Framework Core) cho phép lấy luôn dữ liệu từ các bảng có quan hệ (liên quan) ngay trong lần truy vấn đầu tiên xuống cơ sở dữ liệu.
- Sử dụng thuộc tính điều hướng (Navigation Property) giúp code ngắn gọn và giữ nguyên cấu trúc Object.

```
// Tự động JOIN sang bảng CHU_DE (Lấy tt sách kèm tt chủ đề)
var dsSach = await _context.Sachs .Include(s => s.ChuDes).Where(s
=> s.DonGia > 100000) .ToListAsync();
// Lúc này s.ChuDe.Ten_chu_de đã có dữ liệu!
```

Gom nhóm dữ liệu với `GroupBy()`

❖Nhóm danh sách sách theo mã chủ đề `Mcd`

```
var nhomSach = await _context.Sachs .GroupBy(s => s.Mcd).Select(g => new {
        MaChuDe = g.Key,
        SoLuongSach = g.Count(),
        GiaTrungBinh = g.Average(s => s.Don_gia) }
) .ToListAsync();
```

Gom nhóm lấy tên chủ đề

❖Thống kê tổng số sách của từng Tên Chủ Đề

```
var thongKe = await _context.ChuDes .Select(c => new {
TenChuDe = c.TenChuDe, TongSoSach = c.Sachs.Count(),
TongLuotBan = c.Sachs.Sum(s => (int?)s.SoLuongBan) ?? 0 }) .ToListAsync();
```

Truy vấn CSDL QLBanSach

Lọc theo danh mục:

Lấy danh mục chủ đề có sá

TenChuDe	TongSoSach	TongLuotBan
Tiếng Việt	0	0
Ngoại ngữ	1	7
Công nghệ thông tin	32	1021

Áp dụng LINQ trong controller ASP.NET Core

❖Xử lý Action hiển thị danh sách sản phẩm theo Chủ Đề

```
public async Task<IActionResult> Index(int? mcd) {
        var query =   _context.Sachs.Include(s =>
                      s.ChuDe).AsQueryable();
        if (mcd.HasValue) {
                query = query.Where(s => s.Mcd == mcd.Value);
        }
        var model = await query.ToListAsync();
        return View(model);
}
```

Cảnh báo vấn đề N+1 Query

❖Lỗi hiệu năng phổ biến khi không sử dụng `Include()`

```
var list = _context.Sachs.ToList();
// Mỗi lần chạy câu này sinh thêm 1 câu SELECT xuống DB!
foreach(var item in list) {
        var chuDe = item.ChuDe.Ten_chu_de;
}
```

❖Giải pháp là thay thế câu lệnh truy vấn:

```
// Chỉ chạy duy nhất 1 câu SQL JOIN!
var list = await _context.Sachs.Include(s => s.ChuDe)
.ToListAsync();
```

Tối ưu truy vấn chỉ đọc với `AsNoTracking()`

- Tắt Change Tracker giúp tiết kiệm bộ nhớ RAM và tăng tốc xử lý
- Khi chỉ hiển thị danh sách lên View mà không chỉnh sửa / lưu (`SaveChanges`), bắt buộc dùng `AsNoTracking()`.

```
// Bỏ qua theo dõi trạng thái Entity
var dsSachReadonly = await _context.Sachs
.AsNoTracking().Include(s => s.ChuDe).ToListAsync();
```

Lỗi Multiple enumeration

- Không duyệt lại biến `IQueryable` nhiều lần gây lãng phí SQL
- Chạy 2 câu SQL riêng biệt → Kém hiệu quả

```
var query = _context.Sachs.Where(s => s.Don_gia > 100);
var count = query.Count(); // SQL 1
var first = query.First(); // SQL 2
```

- Giải pháp là:

```
// Thực thi ngay và lưu vào RAM
var list = await _context.Sachs .Where(s => s.Don_gia >
100).ToListAsync();
var count = list.Count;
var first = list.First();
```

Bảng tóm tắt các phương thức LINQ hay dùng

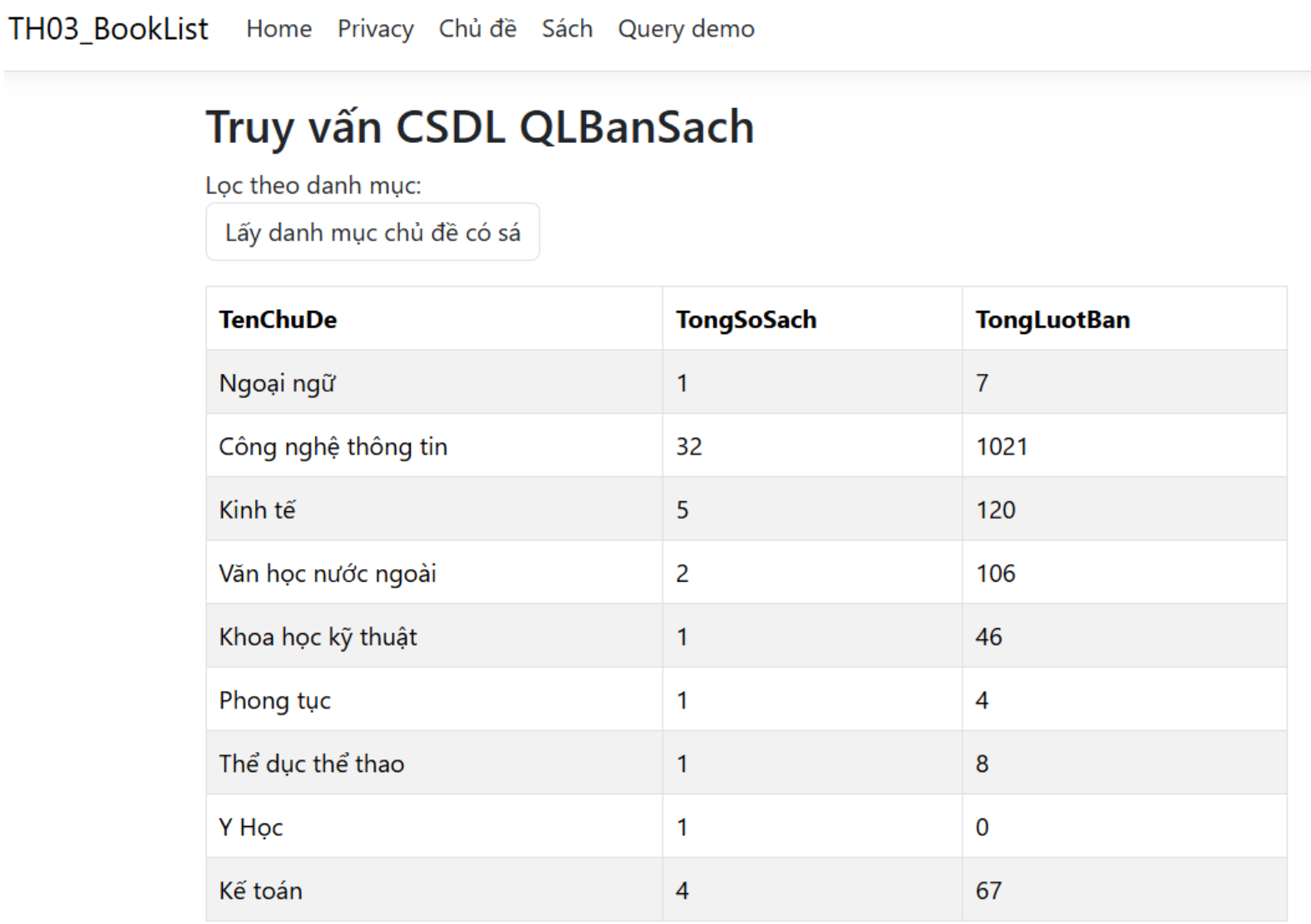
Phương thức	Chức năng tương đương SQL
Where()	Mệnh đề WHERE
Select()	Mệnh đề SELECT (chọn cột/DTO)
OrderBy() / OrderByDescending()	Mệnh đề ORDER BY ASC / DESC
Skip() / Take()	OFFSET ... ROWS FETCH NEXT ... ROWS ONLY
Include()	INNER JOIN / LEFT JOIN theo Navigation Property
GroupBy()	Mệnh đề GROUP BY

Chú ý

- **Luôn dùng Asynchronous:** Gọi `ToListAsync()`, `FirstOrDefaultAsync()` để tránh nghẽn luồng ứng dụng Web.
- **Dùng Any() thay vì Count() > 0**: `Any()` dừng ngay khi gặp 1 bản ghi thỏa mãn, trong khi `Count()` phải đếm toàn bộ bảng.
- **Sử dụng Select DTO**: Tránh nạp những cột dung lượng lớn không cần thiết (như `Mo_ta`, `Hinh_minh_hoa`).

Bài tập

- Xây dựng 1 webpage thử nghiệm các câu lệnh truy vấn phục vụ bài tập lớn



TenChuDe	TongSoSach	TongLuotBan
Ngoại ngữ	1	7
Công nghệ thông tin	32	1021
Kinh tế	5	120
Văn học nước ngoài	2	106
Khoa học kỹ thuật	1	46
Phong tục	1	4
Thể dục thể thao	1	8
Y Học	1	0
Kế toán	4	67